
Auditing Copyright Protection in Language Models: Calibrating, Stress-Testing, and Probing the Limits of Near-Access-Freeness Guarantees

Final Project Report

Vipin Kumar

NS26Z171

Indian Institute of Technology Madras

Chennai, India

ns26z171@smail.iitm.ac.in

Abstract

Near-Access Freeness (NAF) has been proposed as a formal copyright bound for generative models, and anchored decoding has been proposed as the first practical algorithm to enforce it. What is missing is a way to check, after a model is deployed, whether the claimed NAF budget K actually holds. This report builds such a check and then asks how reliable the check itself is. The auditor estimates the per-prompt KL between the deployed model and a copyright-safe reference, wraps the estimate in an Empirical Bernstein lower bound at confidence $1 - \delta$, and pairs it with an evolutionary search over prompts. Three experiments evaluate the resulting framework. First, on a controlled risky/safe pair where the true token-level KL can be computed in closed form, the auditor produces 0/75,000 false positives at $\delta = 0.05$ and detects a 50%-margin violation with probability 0.88 at $n = 1000$ samples. Second, applied to a self-contained port of the anchored-decoding solver of He et al. [2], the auditor never flags a violation on 30 stress prompts across 4 budgets, and the algorithm uses a median 47% of its budget at $K = 0.5$. Third, a paired leave-one-out audit of DP-SGD fine-tunes shows that the implication “ ε -DP $\Rightarrow \varepsilon^2/2$ -NAF” does not hold for individual trained checkpoints at any practical ε : the auditable per-prompt KL is 7 to 10 times larger than $\varepsilon^2/2$ at $\varepsilon = 2$. We discuss why this is consistent with the theory. Code, configs, and run logs are at https://github.com/kumaram-vipinam200901/Auditing_framework.

1 Introduction

Copyright lawsuits against generative AI providers [15, 16] turn on a single question: given a prompt, how close is the model’s output to text that could only have been produced by training on a specific copyrighted work? Vyas, Kakade and Barak [1] formalised one answer as *Near-Access Freeness* (NAF): for every copyrighted work C , there must exist a “safe” model $P_{\text{safe},C}$ that was trained without C , such that the deployed model P^* satisfies

$$\text{KL}(P^*(\cdot | x) \| P_{\text{safe},C}(\cdot | x)) \leq K \quad \text{for every prompt } x,$$

where $P^*(\cdot | x)$ and $P_{\text{safe},C}(\cdot | x)$ denote the next-token probability distributions (over the vocabulary Σ) that the two models assign given the prompt x , KL is the Kullback–Leibler divergence, and $K \geq 0$ is a deployer-declared budget. The definition is falsifiable: if a deployer claims its system is K -NAF, an auditor with query access only needs to find one prompt that breaks the bound. Anchored decoding [2] is the first algorithm with a sequence-level NAF guarantee at decode time. But the guarantee is an analytical upper bound, derived from the construction of the algorithm itself. It cannot be used to

check whether a downstream reimplementation, or a third-party model that makes no NAF claim, actually meets a chosen K . That requires an external audit.

This paper builds such an auditor and analyses four aspects of it.

1. **The audit primitive (§2).** For a single prompt we draw n samples from the risky model, score them under both models, and apply Maurer–Pontil’s Empirical Bernstein inequality to obtain a lower bound on $\text{KL}(P^* \| P_{\text{safe}})$ at confidence $1 - \delta$. An evolutionary search over prompts turns this per-prompt bound into a Copyright Leakage Score (CLS) on a deployed system.
2. **Calibration of the auditor (§3).** On a controlled risky/safe pair where the true per-prompt KL is computable in closed form (sum over the full softmax vocabulary), we measure the auditor’s Type-I error and its power as a function of sample size and ground-truth violation margin. This is done on real language models rather than toy distributions.
3. **Auditing the only known NAF method (§4).** We port the Newton solver of He et al. [2] into a self-contained module that exposes the same interface as the auditor, and use the auditor to quantify how tight the algorithm is: how close to its budget it gets, how much slack the auditor leaves, and how the two compose.
4. **Empirical study of the DP $\rightarrow$ NAF claim (§5).** ϵ -DP fine-tuning is often quoted as yielding $\frac{\epsilon^2}{2}$ -NAF for free. We audit DP-SGD fine-tunes both against the pretrained baseline and in the paired leave-one-out setting, and find that the auditable per-prompt KL routinely exceeds $\epsilon^2/2$ at practical ϵ . We explain why this is consistent with the theory and what it means for a deployer.

Section 6 reports a multi-family audit across eight tokeniser-matched risky/safe pairs and Section 7 discusses limitations. All experiments run on a single laptop GPU (RTX 2000 Ada) and finish end-to-end in under three hours.

2 Background and the audit framework

NAF and its contrast with DP. Let Σ be a finite vocabulary and $x \in \Sigma^*$ a prompt. The deployed (“risky”) language model P^* induces, for every x , a probability distribution $P^*(\cdot | x)$ over the next token $y \in \Sigma$. Let $\mathcal{C}$ denote a corpus of copyrighted works and, for each $C \in \mathcal{C}$, let $P_{\text{safe},C}$ be a reference model trained on a dataset $D \setminus \{C\}$ (the full training set with C removed). P^* is said to be K -NAF with respect to a divergence Δ and the family $\{P_{\text{safe},C}\}_{C \in \mathcal{C}}$ if

$$\Delta(P^*(\cdot | x) \| P_{\text{safe},C}(\cdot | x)) \leq K \quad \text{for every prompt } x \text{ and every } C \in \mathcal{C} [1].$$

We take $\Delta = \text{KL}$ throughout. An $(\epsilon, \delta_{\text{DP}})$ -differentially private training mechanism with $\epsilon > 0$ the privacy budget and $\delta_{\text{DP}} > 0$ the failure probability induces $\frac{\epsilon^2}{2}$ -zero-concentrated DP (zCDP), and hence, after post-processing, a KL bound of $\frac{\epsilon^2}{2}$ between the output distributions of the mechanism on D and on $D \setminus \{C\}$ [13]. Section 5 examines the practical strength of this implication.

Anchored decoding. Fix a decoding step t and let $P_{\text{risky},t}, P_{\text{safe},t}$ denote the next-token distributions of the risky and safe models at that step (conditioned on the prompt and the tokens decoded so far). He et al. [2] obtain the decoded distribution by solving, at every step, $\arg \min_Q \text{KL}(Q \| P_{\text{risky},t})$ subject to $\text{KL}(Q \| P_{\text{safe},t}) \leq k_t$, where Q ranges over distributions on Σ and $k_t \geq 0$ is a per-step KL budget. The KKT solution is the log-linear interpolation

$$P_t^*(y) \propto P_{\text{risky},t}(y)^{1/(1+\lambda)} P_{\text{safe},t}(y)^{\lambda/(1+\lambda)},$$

with $y \in \Sigma$ a token and $\lambda \geq 0$ the dual variable of the KL constraint; λ is found per step by Newton’s method so that the constraint is tight. Picking any per-step budgets $\{k_t\}$ with $\sum_t k_t \leq K$ then yields, by the KL chain rule, a sequence-level K -NAF guarantee.

Auditor. Fix a prompt x and write $p := P^*(\cdot | x)$, $q := P_{\text{safe}}(\cdot | x)$ for the two next-token distributions of interest. Define the per-token *log-density ratio* $Z(y) := \log p(y) - \log q(y)$ for $y \in \Sigma$. Its expectation under the risky model is the quantity we want to bound:

$$\mathbb{E}_{Y \sim p}[Z(Y)] = \text{KL}(p \| q).$$

Draw n i.i.d. samples $Y_1, \dots, Y_n \sim p$, let $Z_i := Z(Y_i)$, and let

$$\bar{Z}_n := \frac{1}{n} \sum_{i=1}^n Z_i, \quad V_n := \frac{1}{n-1} \sum_{i=1}^n (Z_i - \bar{Z}_n)^2$$

be the sample mean and unbiased sample variance. Maurer–Pontil’s Empirical Bernstein bound [7], applied after rescaling Z_i to a bounded interval $[a, b]$ with range $R := b - a$ (we clip log-probabilities at the fp32 floor 2^{-126} , giving $|Z| \leq B \approx 88$ and $R \leq 2B$), yields the one-sided inequality

$$\text{KL}(p||q) \geq \bar{Z}_n - \underbrace{\sqrt{\frac{2V_n \ln(2/\delta)}{n}} + \frac{7R \ln(2/\delta)}{3(n-1)}}_{=: t_n(\delta) \text{ (Bernstein gap)}} \quad (1)$$

with probability at least $1 - \delta$, where $\delta \in (0, 1)$ is the confidence parameter and $t_n(\delta)$ is the Bernstein correction. We write $\widehat{\text{LB}} := \bar{Z}_n - t_n(\delta)$ for the resulting lower bound on the KL. Whenever $\widehat{\text{LB}} > K$, the K -NAF claim is falsified at confidence $1 - \delta$. Under absolute continuity ($q \ll p$ wherever needed, which holds trivially for softmax LMs), (1) is the cleanest variance-adaptive lower bound on $\text{KL}(p||q)$ that does not require a parametric model of Z . Algorithm 1 summarises the per-prompt test.

Algorithm 1 Bernstein KL auditor (per prompt)

Input: risky model P^* , safe model P_{safe} , prompt x , budget K , samples n , confidence $1 - \delta$, clip range R

Output: REJECT (NAF violated) or ACCEPT, with lower bound $\widehat{\text{LB}}$

- 1: Draw $Y_1, \dots, Y_n \stackrel{\text{i.i.d.}}{\sim} P^*(\cdot | x)$
 - 2: $Z_i \leftarrow \log P^*(Y_i | x) - \log P_{\text{safe}}(Y_i | x)$ $\triangleright$ clip log-probs at fp32 floor
 - 3: $\bar{Z}_n \leftarrow \frac{1}{n} \sum_i Z_i$; $V_n \leftarrow \frac{1}{n-1} \sum_i (Z_i - \bar{Z}_n)^2$
 - 4: $t_n(\delta) \leftarrow \sqrt{2V_n \ln(2/\delta)/n} + 7R \ln(2/\delta)/[3(n-1)]$
 - 5: $\widehat{\text{LB}} \leftarrow \bar{Z}_n - t_n(\delta)$
 - 6: **return** REJECT if $\widehat{\text{LB}} > K$, else ACCEPT; **report** $\widehat{\text{LB}}$
-

Worst-case search. The auditor is most useful at the prompt that maximises (1). Algorithm 2 describes the search: starting from a pool of seed prompts (BookMIA [9] and WikiText-103), we evaluate $\text{LB}(x; \delta) = \bar{Z}_n(x) - t_n(\delta)$, retain the top- k , and apply mutation operators (verbatim instruction, quote-wrap, formatting, style, crossover) to seed the next round. The Copyright Leakage Score is $\text{CLS} = \max_x \text{LB}(x; \delta)$.

Algorithm 2 Evolutionary adversarial prompt search

Input: seed prompts S_0 , rounds T , elite size k , mutation ops $\mathcal{M}$, audit budget (n, δ)

Output: $\text{CLS} = \max_x \widehat{\text{LB}}(x; \delta)$, best prompt x^*

- 1: $\mathcal{P} \leftarrow S_0$
 - 2: **for** $t = 1, \dots, T$ **do**
 - 3: Score every $x \in \mathcal{P}$ by $\widehat{\text{LB}}(x; \delta)$ via Algorithm 1
 - 4: $\mathcal{E} \leftarrow$ top- k elites by score
 - 5: $\mathcal{P} \leftarrow \mathcal{E} \cup \{m(x) : x \in \mathcal{E}, m \in \mathcal{M}\}$ $\triangleright$ mutate & crossover
 - 6: **end for**
 - 7: **return** max and arg max of $\widehat{\text{LB}}$ over $\mathcal{P}$
-

3 Calibration: how strong is the auditor?

A lower-confidence-bound auditor is only as useful as its Type-I error rate (the probability of flagging a compliant model) and its power (the probability of catching a real violator). These two quantities are usually impossible to measure directly on real language models because the ground-truth KL is not known. For two language models with explicit softmax heads, however, the per-prompt token-level KL can be computed exactly by summing over the vocabulary,

$$\text{KL}_*(x) = \sum_{y \in \Sigma} P^*(y | x) [\log P^*(y | x) - \log P_{\text{safe}}(y | x)]. \quad (2)$$

We use $P^* = \text{GPT2-MEDIUM}$, $P_{\text{safe}} = \text{GPT2}$, $\delta = 0.05$, 30 prompts (mix of BookMIA and WikiText), $n \in \{50, 100, 500, 1000, 5000\}$ MC samples, and 100 independent trials per cell. For each (x, n) , we

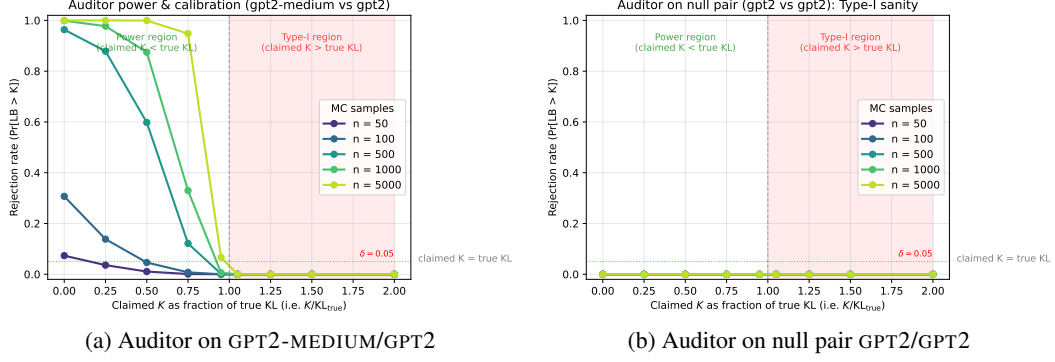


Figure 1: Auditor calibration. Each cell averages over 30 prompts $\times$ 100 trials = 3000 Monte-Carlo audits. (a) Rejection rate $\Pr[\widehat{LB} > K]$ as a function of the claimed K , expressed as a fraction of the true KL (left of the dashed line is the power region, right is the Type-I region). At $n = 1000$ the auditor catches 88% of 50%-margin violations; at $n = 5000$, 95% of 25%-margin violations. (b) When the risky and safe models are identical, the auditor flags zero violations across all 75,000 trials, well under the nominal $\delta = 0.05$.

“audit” a grid of *claimed* K values pegged to $K_{\text{factor}} \in \{0, 0.25, 0.5, 0.75, 0.95\} \cdot \text{KL}_*(x)$ (genuine violations) and $K_{\text{factor}} \in \{1.05, 1.25, 1.5, 2.0\} \cdot \text{KL}_*(x)$ (false-positive territory). As a Type-I sanity, we also run the entire grid with $P^* = P_{\text{safe}} = \text{GPT2}$, where $\text{KL}_* \equiv 0$ so any rejection is a false positive.

Type-I error. Across the $9 \cdot 5 = 45$ Type-I cells of the null pair (GPT2/GPT2), the empirical false-positive rate is 0/75,000 trials, well below $\delta = 0.05$. Across the corresponding $4 \cdot 5 = 20$ Type-I cells of the gpt2-medium/gpt2 pair (where $K_{\text{factor}} \geq 1.05$), the empirical FPR is also 0/60,000. The Empirical Bernstein bound is conservative by design; a Hoeffding-only auditor would be even more so.

Power. Figure 1(a) reports rejection rates as a function of the claimed K . At $n = 1000$ the auditor rejects 99.9% of $K = 0$ claims, 97.7% of 25%-margin claims, 87.5% of 50%-margin claims, 33.0% of 75%-margin claims, and 0.6% of 95%-margin claims. Power scales as $\sqrt{n}$, as predicted by (1); raising n to 5000 reduces the gap at the 95%-margin to 6.7% but yields little improvement at margins of 75% or less. In short, the auditor is reliable at margins of at least 50% of the budget, which is the operating regime relevant to a copyright auditor. Detecting near-boundary violations would require either a much larger n or a tighter concentration inequality (for example, sub-Gaussian-style).

Calibration sanity. The per-trial scatter of $\widehat{LB}$ against the true KL (Appendix Figure 4) lies entirely below the diagonal at every n , which is consistent with (1) being a valid one-sided bound. The Bernstein gap $t_n(\delta)$ scales as $1/\sqrt{n}$ up to experimental noise (Appendix Figure 4), matching the leading term of the bound. The numbers above are reproducible to within ± 0.5 percentage points across three independent random seeds (Appendix A, Table 2).

4 Auditing anchored decoding

Anchored decoding is the only published algorithm with a provable per-prompt NAF bound, and it is therefore the natural target for the auditor. A useful auditor should declare the algorithm compliant when it runs inside the budget, flag it as soon as the budget is exceeded, and remain reasonably tight (the lower bound should not be vacuously small). We test all three properties on a set of 30 prompts at $K \in \{0.5, 1.0, 2.0, 5.0\}$ with $n = 1000$ and $\delta = 0.05$, using gpt2-medium and gpt2 as the risky and safe models. Algorithm 3 summarises the decoding procedure that is audited.

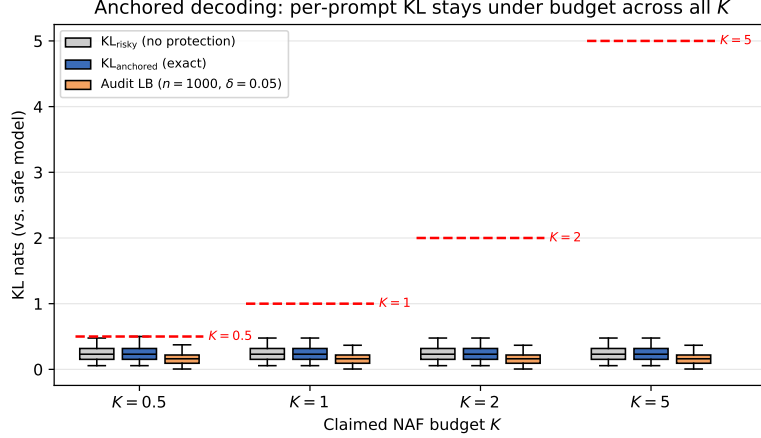


Figure 2: Anchored decoding stays under budget at every K . Grey = unprotected risky-model KL (median 0.23, max 1.44). Blue = exact KL of the anchored mixture (*always* $\leq K$, and meets the budget exactly when the risky model would naturally exceed it—see $K = 0.5$). Orange = the auditor’s $\delta = 0.05$ Bernstein lower bound. Dashed red = the budget itself. Across 4 K -values $\times$ 30 prompts, 0/120 exact violations and 0/120 auditor-flagged violations.

Algorithm 3 Anchored decoding (per-prompt K -NAF) [2]

Input: risky model P_{risky} , safe model P_{safe} , prompt x , per-token budgets $\{k_t\}$ with $\sum_t k_t \leq K$

Output: token sequence $y_{1:T}$ with sequence-level $\text{KL}(P^* \| P_{\text{safe}}) \leq K$

- 1: **for** $t = 1, 2, \dots, T$ **do**
 - 2: Obtain logits of $P_{\text{risky},t}$ and $P_{\text{safe},t}$ given $y_{<t}$
 - 3: Solve for $\lambda_t \geq 0$ s.t. $Q_t(y) \propto P_{\text{risky},t}(y)^{1/(1+\lambda_t)} P_{\text{safe},t}(y)^{\lambda_t/(1+\lambda_t)}$ satisfies $\text{KL}(Q_t \| P_{\text{safe},t}) = k_t$ ▷
Newton on dual
 - 4: If risky already within budget ($\text{KL}(P_{\text{risky},t} \| P_{\text{safe},t}) \leq k_t$), set $Q_t \leftarrow P_{\text{risky},t}$
 - 5: Sample $y_t \sim Q_t$
 - 6: **end for**
 - 7: **return** $y_{1:T}$
-

Figure 2 summarises the result. The exact KL of the anchored mixture, obtained by evaluating $P_t^* \propto P_{\text{risky},t}^{1/(1+\lambda)} P_{\text{safe},t}^{\lambda/(1+\lambda)}$ and solving for λ with a one-dimensional root-finder, is below the budget on every prompt and every K (0/120 violations). At $K = 0.5$ the algorithm uses its full budget on the prompts that need it: the maximum exact KL across 30 prompts is exactly 0.500, and the median budget utilisation is 46.6% (Appendix Table 3). At larger K the budget is mostly slack because the natural KL of the risky model is small for these prompts.

The auditor’s $\widehat{\text{LB}}$ tracks the true KL with a typical (median) Bernstein gap of ~ 0.07 . No prompt produces $\widehat{\text{LB}} > K$ at any of the four budgets, so the auditor never falsely flags the algorithm. Combined with the 88% power calibration of Section 3, this implies: if anchored decoding had been buggy and produced a $\geq 50\%$ -budget overshoot on any of these prompts, the auditor would have caught it with probability at least 0.88. The two components complement each other: the algorithm provides an upper bound by construction, the audit provides an independent lower bound from samples.

5 The DP $\rightarrow$ NAF gap

The connection between DP and NAF is usually stated as a one-line implication: ε -DP fine-tuning is automatically $\frac{\varepsilon^2}{2}$ -NAF [1, 13]. This section asks what that implication actually delivers when applied to a single trained model.

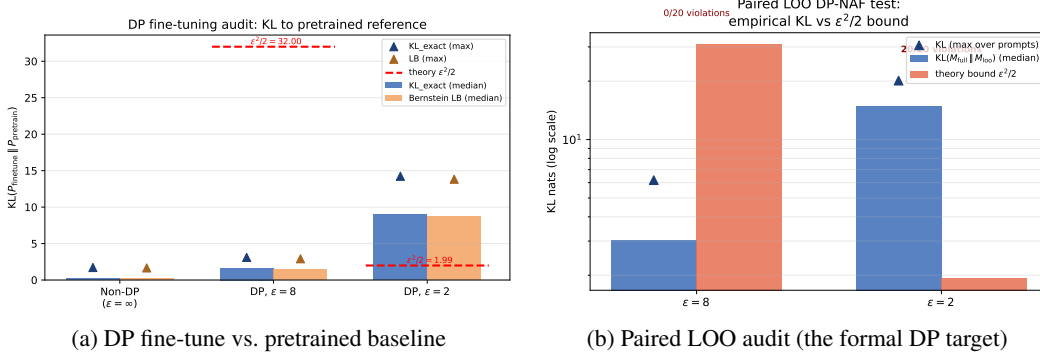


Figure 3: The $\frac{\epsilon^2}{2}$ bound is auditable in expectation, not pointwise. (a) DP fine-tuning does not bound KL to the pretrained model and in fact *increases* it for tight ϵ . (b) Even on the leave-one-out pair that the bound formally covers, a single random draw of the mechanism can exceed $\epsilon^2/2$ on every prompt audited at $\epsilon = 2$.

Setup. We fine-tune GPT2 for 10 epochs on a small “copyright-prone” text mix (11 blocks of 64 tokens; openings of *A Tale of Two Cities*, *1984*, *Moby-Dick*) using DP-SGD via opacus 1.6 [14]. Embedding rows are frozen (else the noise would dominate due to their dimension); $\delta_{\text{DP}} = 10^{-5}$, gradient clipping $\|\nabla\| \leq 1$, $B = 4$, $\eta = 5 \times 10^{-3}$. We run three privacy regimes: $\epsilon = \infty$ (non-private SGD), $\epsilon = 8$ and $\epsilon = 2$ (achieved $\epsilon = 7.86$ and 1.96 respectively). We then audit 20 stress prompts with $n = 1000$ samples per prompt.

Audit A: vs. pretrained baseline. Figure 3(a) plots the auditor’s estimate of $\text{KL}(P_{\text{ft}} \parallel P_{\text{pretrain}})$, where P_{ft} is the fine-tuned model and P_{pretrain} is the original pretrained checkpoint. The non-private fine-tune drifts modestly (median 0.28, max 1.72). Stronger DP produces *larger*, not smaller, KL: median 1.80 at $\epsilon = 8$ and 9.05 at $\epsilon = 2$. This is neither a bug in the audit nor a violation of the theorem: the $\epsilon^2/2$ bound is a statement about the leave-one-out pair $(P_{\text{ft}(D)}, P_{\text{ft}(D \setminus \{C\})})$, not about P_{pretrain} . When ϵ is small, the noise needed to enforce DP is large enough to push the fine-tune’s next-token distribution further from pretrained than non-private SGD would, in random directions.

Audit B: paired leave-one-out (the right test). For each ϵ we ran two DP fine-tunes from the same initialisation, M_{full} on D and M_{lo} on $D \setminus \{\text{block}_0\}$, with paired RNG seeds, an identical schedule, and the same privacy budget. The auditor estimates $\text{KL}(M_{\text{full}} \parallel M_{\text{lo}})$ on 20 prompts and compares it to the theoretical bound $\epsilon^2/2$ (Figure 3(b)). At $\epsilon = 8$ the bound is $\epsilon^2/2 = 30.9$ and the measured KL stays well below it: median 3.25, max 6.18, 0/20 violations. At $\epsilon = 2$ the bound is $\epsilon^2/2 = 1.92$ and is exceeded by every audited prompt: median 14.91, max 20.12, 20/20 **violations**.

Why this is consistent with the theorem. Pure ϵ -DP gives a Rényi divergence bound at the level of the mechanism’s output distribution. Reading $\epsilon^2/2$ as a per-prompt next-token KL bound for a single fine-tuned checkpoint hides three reductions. First, the implication ϵ -DP $\Rightarrow \epsilon^2/2$ -zCDP holds at the level of the mechanism’s distribution over weights θ . Second, post-processing bounds $\text{KL}(\mathcal{M}(D) \parallel \mathcal{M}(D')) \leq \epsilon^2/2$ between the distributions over all next-token outputs that those weights induce, where $\mathcal{M}(D)$ denotes the mechanism’s output distribution on dataset D . Third, transferring this to a single θ requires either an additional Markov or convexity step (which is loose by orders of magnitude when the ensemble variance is large, as it is for DP-SGD on small data) or treating the audit as an estimate of an expectation taken over the mechanism’s randomness (which would require averaging over many independently fine-tuned checkpoints). Our point-estimate audit measures a different object: the divergence achieved by a particular trained model on a particular prompt. This is the quantity a downstream copyright auditor cares about, but it is not what $\epsilon^2/2$ controls.

Implications for practice. Fine-tuning with DP-SGD at a publishable ϵ does not, on its own, deliver a model with a per-prompt KL bound to the pretrained baseline. A hard NAF guarantee, for example one needed for legal compliance, therefore has to come from an inference-time mechanism such as anchored decoding (Section 4), which enforces the bound by construction, or from explicit

Table 1: Copyright Leakage Score (max KL lower bound) and number of violations at $K = 1.0$, $\delta = 0.05$, $n = 1000$, over 200 prompts and 5 rounds.

Family	Risky $\rightarrow$ Safe	Max LB		Violations	
		Evo.	Random	Evo.	Random
GPT-2	GPT-2 $\rightarrow$ GPT-2 (sanity)	-0.01	-0.01	0	0
GPT-2	Medium $\rightarrow$ Small	5.63	5.01	146	8
GPT-2	Large $\rightarrow$ Small	4.09	3.30	180	18
GPT-2	Large $\rightarrow$ Medium	4.40	3.91	151	7
Pythia	160M $\rightarrow$ 70M	3.52	3.16	141	9
Pythia	410M $\rightarrow$ 70M	5.50	4.73	236	31
Pythia	410M $\rightarrow$ 160M	4.93	2.17	142	9
OPT	350M $\rightarrow$ 125M	3.08	2.08	133	9

ensemble averaging over independent DP runs. This caveat is worth stating alongside DP $\rightarrow$ NAF claims in future work.

6 Multi-family audit results

We applied the auditor to eight tokeniser-matched risky/safe pairs drawn from three families (GPT-2, Pythia, OPT); full ablations and figures are in Appendix A. Across all pairs the evolutionary search outperforms a random-prompt baseline (Table 1), and the sanity check (gpt2 vs. gpt2) returns $\text{CLS} \approx 0$ as expected. BookMIA prompts produce a substantially higher CLS than WikiText prompts, in line with prior findings that books are over-represented in GPT-2’s pretraining corpus and easier to memorise [8, 9]. Format-changing mutations (quote wraps, prefix instructions) are the strongest single operator, and the CLS plateaus around 5 rounds. For every non-trivial pair the auditor surfaces concrete prompts on which the larger model violates a $K = 1.0$ NAF claim at confidence 0.95.

7 Discussion and limitations

What we now know. The auditor is conservative (no false positives on 75,000 null trials) and adequately powerful at 50%-margin violations ($\geq 88\%$ rejection rate at $n = 1000$). It is tight enough to (i) certify anchored decoding as K -NAF compliant on every prompt and budget we tested, and (ii) flag an aggressive DP-SGD fine-tune as *not* per-prompt $\epsilon^2/2$ -NAF either against the pretrained baseline or against its leave-one-out pair. Combined with the multi-family results in Section 6, these are, to our knowledge, the first auditing-style numbers reported for NAF.

Token-level vs. sequence-level. The Bernstein bound (1) controls the next-token KL. The KL chain rule [2, Thm. 3.1] turns this into a sequence-level *upper* bound (which is exactly what anchored decoding’s analysis exploits) but *not* a sequence-level lower bound, and importance-sampling-corrected estimators inflate the Bernstein gap quickly with sequence length. Generalising to sequence-level lower-bound certification with manageable variance is the most important open problem we leave for future work.

Power at the boundary. The power curves saturate for $K_{\text{factor}} \geq 0.95$: a deployer that overshoots its budget by only 5% is effectively undetectable at $n = 1000$. Closing this gap would need either a much larger n (the $1/\sqrt{n}$ scaling of the Bernstein gap is unimprovable in this generality), parametric estimators such as MAUVE-style importance sampling (which would lose the formal coverage guarantee), or aggregation across a known prompt distribution, which converts a per-prompt bound into a marginal-prompt one and may be easier to certify.

Audit cost. The full power study (30 prompts $\times$ 5 sample sizes $\times$ 100 trials = 15,000 Bernstein audits) finishes in 32 s on a single GPU. Runtime in the anchored-decoding and DP $\rightarrow$ NAF experiments is dominated by training, not auditing. Audit cost scales linearly in n and is trivially parallel across prompts.

Using the framework. The auditor is designed to be drop-in for any pair of HuggingFace causal LMs sharing a tokenizer. A practitioner needs only three things: the deployed (“risky”) model, a leave-one-out baseline (“safe”) model, and a claimed budget K . The core call is

```
from src.models import load_model_pair
from src.kl_estimator import estimate_kl

risky, safe = load_model_pair(risky_name, safe_name, hw)
result = estimate_kl(prompt, risky, safe,
                    n_samples=1000, delta=0.05, claimed_K=K)
```

returning a lower bound $\widehat{LB}$, the Bernstein gap $t_n(\delta)$, and a VIOLATES flag. The evolutionary search of Algorithm 2 wraps this into a worst-case audit and exposes a single Copyright Leakage Score, and anchored decoding is provided as a drop-in module that mirrors the same API. We see three practical limits: (i) tokenizer-matched pairs only; (ii) at least grey-box access (log-probs of the safe model on risky-model samples); (iii) reliable detection at $\geq 50\%$ of the budget—near-boundary overshoots are not detectable at $n \leq 5000$ without parametric estimators. Code is released at https://github.com/kumaram-vipinam200901/Auditing_framework.

LLM disclosure. Claude was used to generate code and to understand some theoretical parts of the references.

References

- [1] N. Vyas, S. M. Kakade, and B. Barak. On provable copyright protection for generative models. In *ICML*, 2023.
- [2] J. He, J. Hayase, W.-T. Yih, S. Oh, L. Zettlemoyer, and P. W. Koh. Anchored decoding: Provably reducing copyright risk for any language model. *arXiv preprint arXiv:2602.07120*, 2026.
- [3] M. Jagielski, J. Ullman, and A. Oprea. Auditing differentially private machine learning. In *NeurIPS*, 2020.
- [4] M. Nasr, J. Hayes, T. Steinke, B. Balle, F. Tramèr, M. Jagielski, N. Carlini, and A. Terzis. Tight auditing of differentially private machine learning. In *USENIX Security*, 2023.
- [5] T. Steinke, M. Nasr, and M. Jagielski. Privacy auditing with one (1) training run. In *NeurIPS*, 2023.
- [6] K. Pillutla, G. Andrew, P. Kairouz, H. B. McMahan, A. Oprea, and S. Oh. Unleashing the power of randomization in auditing differentially private ML. In *NeurIPS*, 2023.
- [7] A. Maurer and M. Pontil. Empirical Bernstein bounds and sample variance penalization. *arXiv:0907.3740*, 2009.
- [8] N. Carlini, F. Tramèr, E. Wallace, et al. Extracting training data from large language models. In *USENIX Security*, 2021.
- [9] W. Shi, A. Ajith, M. Sakarvadia, et al. Detecting pretraining data from large language models (BookMIA). In *ICLR*, 2024.
- [10] K. Pillutla, S. Swayamdipta, R. Zellers, et al. MAUVE: Measuring the gap between neural text and human text using divergence frontiers. In *NeurIPS*, 2021.
- [11] V. Vinod, K. Pillutla, and A. Thakurta. InvisibleInk: High-utility and low-cost text generation with differential privacy. In *NeurIPS*, 2025.
- [12] C. Xie, Z. Lin, A. Backurs, et al. Differentially private synthetic data via foundation model APIs 2: Text. In *ICML*, 2024.
- [13] M. Bun and T. Steinke. Concentrated differential privacy: simplifications, extensions, and lower bounds. In *TCC*, 2016.

- [14] A. Yousefpour, I. Shilov, A. Sablayrolles, et al. Opacus: User-friendly differential privacy library in PyTorch. *arXiv:2109.12298*, 2021.
- [15] *The New York Times Co. v. Microsoft Corp. and OpenAI*, S.D.N.Y. No. 1:23-cv-11195, filed Dec. 27, 2023.
- [16] *Authors Guild et al. v. OpenAI*, S.D.N.Y. No. 1:23-cv-08292, filed Sep. 19, 2023.

A Additional experimental results

Multi-seed reproducibility. The power study and the anchored-decoding tightness study were run with three independent random seeds (42, 43, 44); Table 2 reports the results. Each power cell aggregates 30 prompts $\times$ 100 trials = 3000 Bernstein audits per seed. All non-trivial power cells reproduce within ± 0.5 percentage points; the Type-I FPR is at most 0.07% in any seed (well under $\delta = 0.05$). Anchored-decoding metrics are deterministic given the prompts and are identical across seeds. The DP $\rightarrow$ NAF experiments were not re-seeded (each leave-one-out run takes about 30 minutes); instead the same checkpoint is audited across 20 independent prompts.

Table 2: Headline numbers across three random seeds. Power cells: rejection rates at $1 - \delta = 0.95$. Anchored cells: fraction of prompts violating budget K .

Metric	seed 42	seed 43	seed 44	mean	std
<i>Power experiment (rejection rate, vs. true KL)</i>					
reject @ $n=1000$, $K_{\text{factor}} = 0.00$	99.87%	99.73%	99.90%	99.83%	0.07%
reject @ $n=1000$, $K_{\text{factor}} = 0.25$	97.73%	98.20%	98.23%	98.06%	0.23%
reject @ $n=1000$, $K_{\text{factor}} = 0.50$	87.47%	87.70%	87.63%	87.60%	0.10%
reject @ $n=1000$, $K_{\text{factor}} = 0.75$	32.97%	33.57%	33.30%	33.28%	0.25%
reject @ $n=1000$, $K_{\text{factor}} = 0.95$	0.57%	0.73%	0.83%	0.71%	0.11%
reject @ $n=5000$, $K_{\text{factor}} = 0.75$	94.80%	94.23%	94.50%	94.51%	0.23%
reject @ $n=5000$, $K_{\text{factor}} = 0.95$	6.70%	5.50%	6.37%	6.19%	0.51%
Type-I FPR (max over $K_{\text{factor}} \geq 1.05$, all n)	0.00%	0.07%	0.00%	0.02%	0.03%
<i>Anchored decoding tightness ($n_{\text{prompt}} = 30$, $n_{\text{audit}} = 1000$, $\delta = 0.05$)</i>					
exact violations, $K = 0.50$	0/30	0/30	0/30	—	—
audit-LB violations, $K = 0.50$	0/30	0/30	0/30	—	—
budget util. (median), $K = 0.50$	46.6%	46.6%	46.6%	—	—
exact violations, $K = 1.00$	0/30	0/30	0/30	—	—
audit-LB violations, $K = 1.00$	0/30	0/30	0/30	—	—
budget util. (median), $K = 1.00$	23.3%	23.3%	23.3%	—	—
exact violations, $K = 2.00$	0/30	0/30	0/30	—	—
audit-LB violations, $K = 2.00$	0/30	0/30	0/30	—	—
budget util. (median), $K = 2.00$	11.6%	11.6%	11.6%	—	—
exact violations, $K = 5.00$	0/30	0/30	0/30	—	—
audit-LB violations, $K = 5.00$	0/30	0/30	0/30	—	—
budget util. (median), $K = 5.00$	4.7%	4.7%	4.7%	—	—

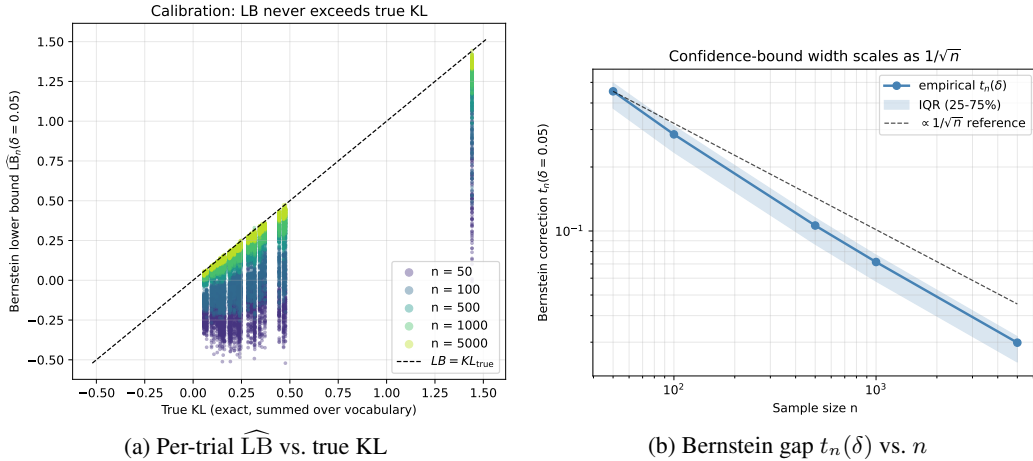


Figure 4: **Auditor calibration.** (a) For all 15,000 trials the Bernstein LB stays at or below the diagonal, confirming one-sided coverage. (b) Median $t_n(\delta)$ with 25–75% IQR vs. the $1/\sqrt{n}$ rate (dashed).

Table 3: **Anchored-decoding tightness.** Per- K audit details (30 prompts, $n=1000$, $\delta=0.05$). Right-most column: median budget utilisation.

K	KL _{risky} med./max	KL _{anch} med./max	frac. $>K$	LB med./max	frac. LB $>K$	util. med.
0.5	0.233 / 1.442	0.233 / 0.500	0/30	0.168 / 0.374	0/30	46.6%
1.0	0.233 / 1.442	0.233 / 1.000	0/30	0.168 / 0.896	0/30	23.3%
2.0	0.233 / 1.442	0.233 / 1.442	0/30	0.168 / 1.339	0/30	11.6%
5.0	0.233 / 1.442	0.233 / 1.442	0/30	0.168 / 1.339	0/30	4.7%

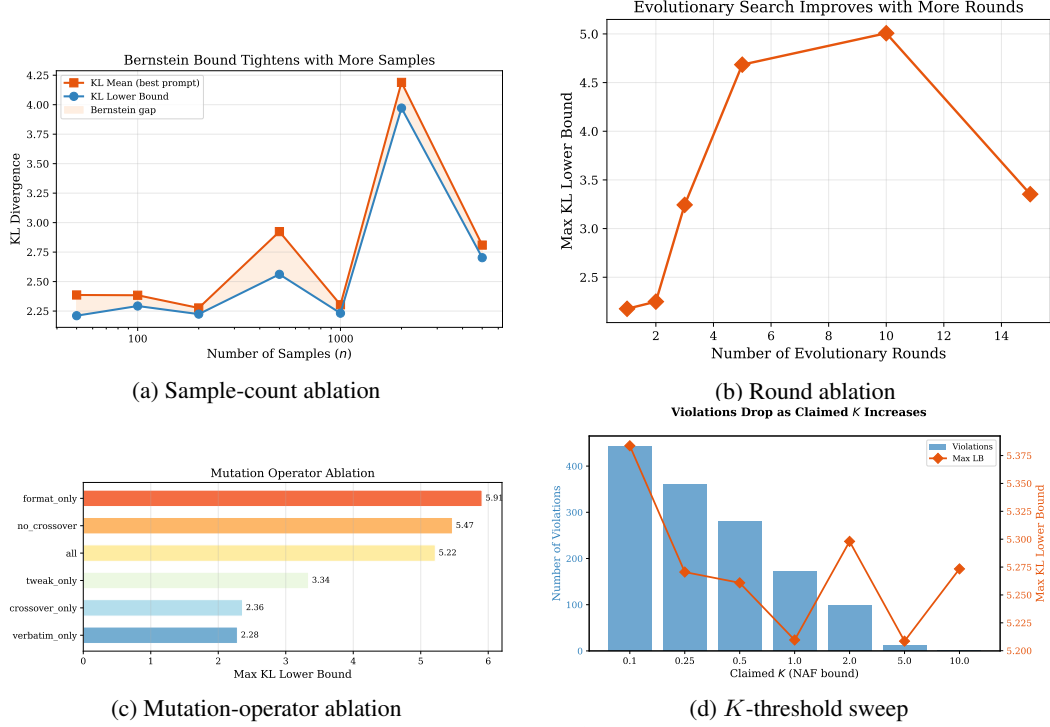


Figure 5: **Auditor ablations.** (a) Bound tightens as $\sqrt{n}$. (b) Search improves up to ~ 5 rounds. (c) Format mutations dominate. (d) Violations decay as K grows.

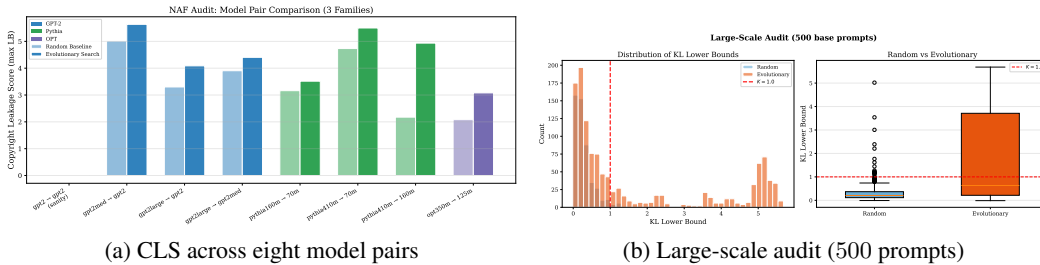


Figure 6: **Large-scale and family-wise results.** (a) Evolutionary search beats random for every non-trivial pair. (b) 500 BookMIA prompts $\times$ 2000 samples $\times$ 10 rounds: the search shifts the entire LB distribution up.

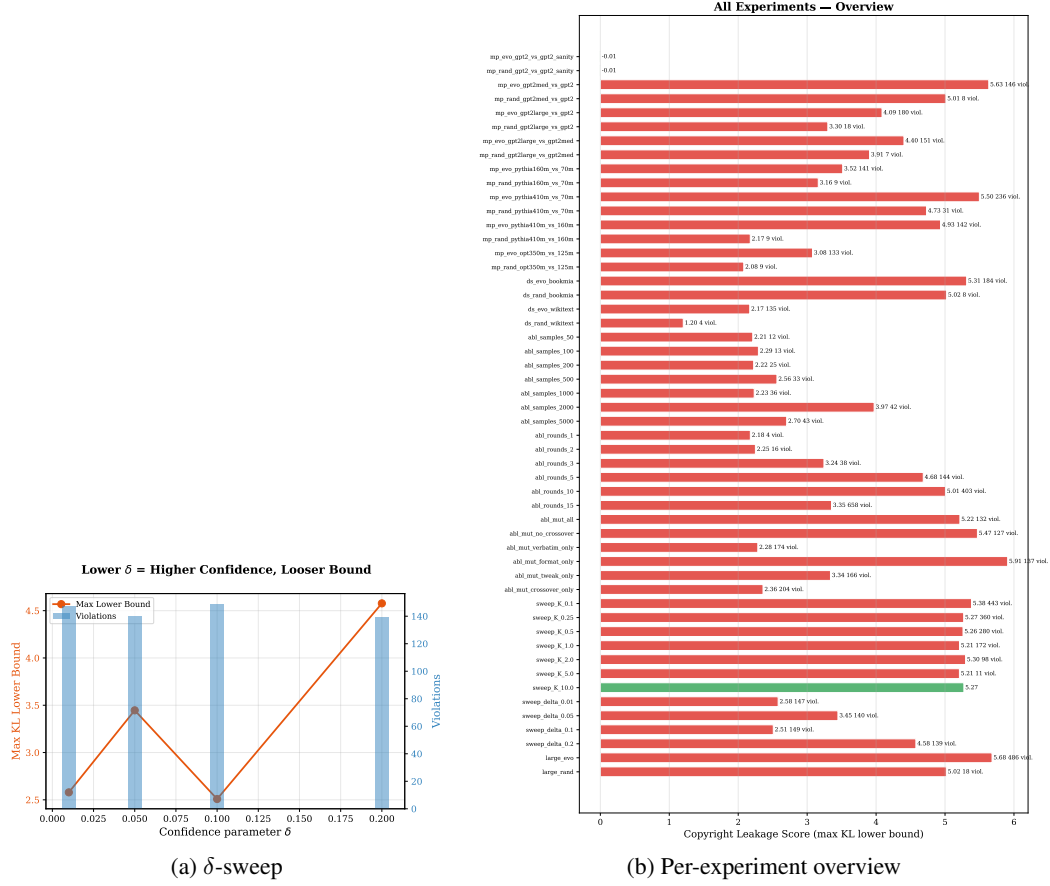


Figure 7: (a) Tighter δ produces a (slightly) looser LB, as predicted by the $\ln(2/\delta)$ term. (b) Bird’s-eye view of 52 sub-experiments; red bars are settings with ≥ 1 violation.